Implementation of a Secure Web-Based Chat Application using SSL/TLS

An Innovative Assignment for the subject of CNS

Submitted by:

Name: Trishna Das

Registration no.: 2302041055

Semester: 5th

Subject name: Cryptographic Foundation and Network Security (CNS)

Department Of Computer Science & Engineering

Veer Surendra Sai University Of Technology, Burla

Submitted To :

Dr. Sucheta Panda

Assistant Professor,

Department Of Computer Science & Engineering,

VSSUT, Burla

TABLE OF CONTENTS

Implementation of a Secure Web-Based Chat Application using SSL/TLS.....1
1. Abstract.....2
1.1 Introduction..... 2
2. Objectives.....3
3. Methodology / Implementation.....4
3.1 System Architecture and Design..... 4
3.2 Architectural Flowchart.....4
3.4 Implementation Steps & Code..... 5
4. Results..... 7
4.1 Browser-Based Verification.....7
4.2 Network-Level Analysis with Wireshark..... 8
5. Conclusion..... 9
5.1 Conclusion.....9
6. Future Scope.....10
6.1 Future Scope.....10
6. References.....10

1. Abstract

This report details the design, implementation, and security validation of a real-time, web-based chat application. The project's central focus is the practical application of the Transport Layer Security (TLS) protocol to secure all data transmitted between clients and the server. By establishing an encrypted communication channel, the application protects against common network threats such as eavesdropping and man-in-the-middle (MITM) attacks, which are inherent risks when using standard, unencrypted protocols. The main goal of this project is to build a tangible, working system that demonstrates the three fundamental security guarantees of TLS: Confidentiality (through encryption), Integrity (through message authentication codes), and Authentication (through the use of digital certificates).

1.1 Introduction

This project specifically achieves its security objectives by implementing a WebSocket Secure (WSS) server, which layers the standard WebSocket protocol over a secure TLS connection. This approach provides a practical, hands-on demonstration of secure communication in a modern web environment. The subsequent sections of this report will elaborate on the system's objectives, the methodology and implementation details including architecture and code snippets, and the verification of the secure communication channel through both browser-based and network-level analysis.

2. Objectives

- To design and implement a functional client-server architecture for a real-time chat application using the WebSocket protocol for low-latency, bidirectional communication.
- To secure the entire communication channel by integrating SSL/TLS, upgrading the standard ws:// protocol to the encrypted wss:// protocol.
- To configure and deploy a backend server using Node.js that serves the client application over HTTPS and manages secure WebSocket connections simultaneously.[8]
- To perform security analysis and verification to confirm that the data channel is successfully encrypted, using both browser-based developer tools and low-level network packet inspection.
- The choice of WebSockets is critical for the real-time nature of a chat application, as it provides a persistent, full-duplex communication channel over a single TCP connection, avoiding the high overhead of traditional HTTP polling. The security is layered on top by using the WSS protocol, which mandates that the initial connection handshake and all subsequent data transfer occur within a TLS-encrypted tunnel.

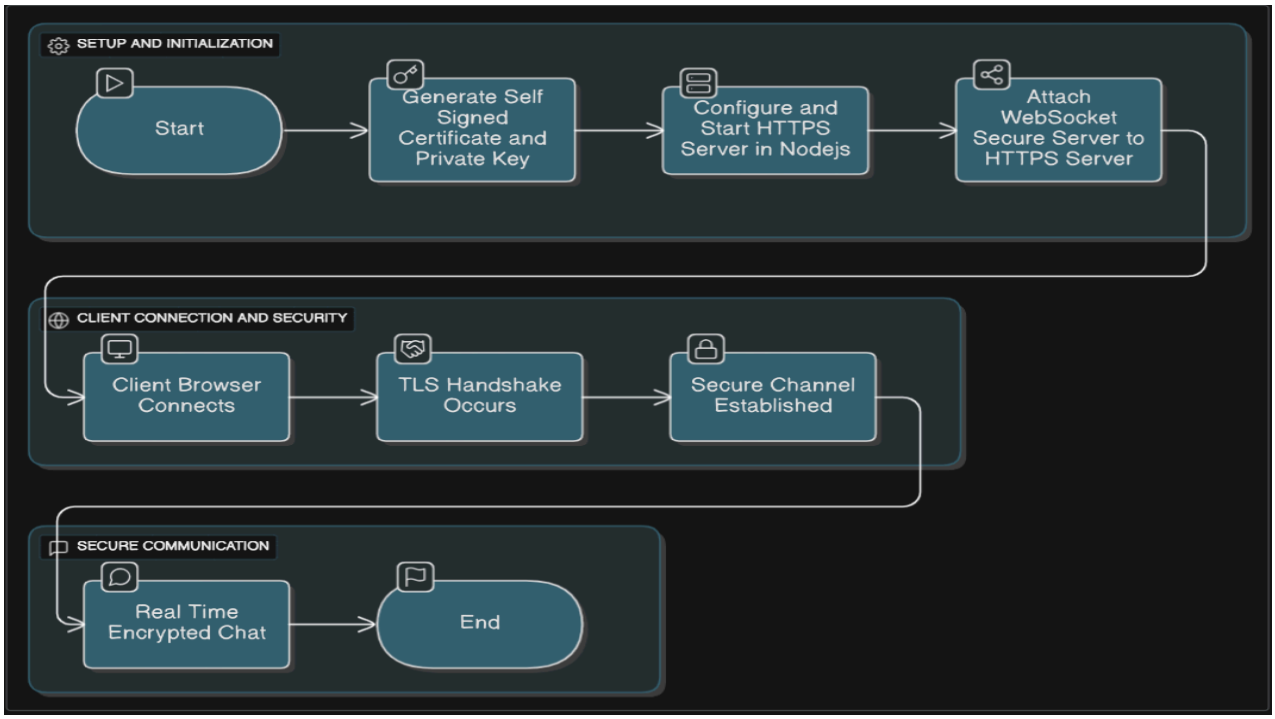
3. Methodology

3.1 System Architecture and Design

The application is built on a client-server model designed for real-time interaction.

- **Server:** A backend application built with Node.js. It has two primary responsibilities:
  1. **HTTPS Server:** To securely serve the static client-side files (HTML, CSS, JavaScript) to the user's browser.
  2. **WebSocket Secure (WSS) Server:** To manage persistent, real-time connections with multiple clients, broadcasting incoming messages to all other connected participants.
- **Client:** A simple web page that runs in any modern browser. Its JavaScript code establishes a wss:// connection to the server and handles the sending and receiving of chat messages.

3.2 Architectural Flowchart



Secure Communication using SSL/TLS(Web App Flow)

3.3 Tools and Libraries

- **Runtime Environment: Node.js** was chosen for its event-driven, non-blocking I/O model, which is highly efficient for handling numerous concurrent WebSocket connections.
- **Web Framework: Express.js** is used as a minimal framework to simplify the process of serving the initial client-side HTML file over HTTPS.

- **WebSocket Library:** The **ws** library is a popular, high-performance WebSocket implementation for Node.js, used to create the WebSocket server.
- **Cryptography Tool: OpenSSL** is the industry-standard tool used to generate the self-signed X.509 certificate and private key required for the server to establish TLS connections.

3.4 Implementation Steps & Code

Step 1: Generating a Self-Signed Certificate

For development, a self-signed certificate is sufficient. The following OpenSSL command was used to generate the necessary files:

openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -sha256 -days 365 -nodes

- req: Command for certificate signing request (CSR) and certificate generation.
- -x509: Specifies the output should be a self-signed certificate.
- -newkey rsa:2048: Generates a new 2048-bit RSA private key.
- -keyout key.pem: The filename for the private key.
- -out cert.pem: The filename for the certificate.
- -nodes: Prevents encrypting the private key with a passphrase, simplifying server startup for development.

Step 2: Building the Secure Server

The server code integrates an HTTPS server and a WSS server to run on the same port. The following code snippet illustrates the core implementation.

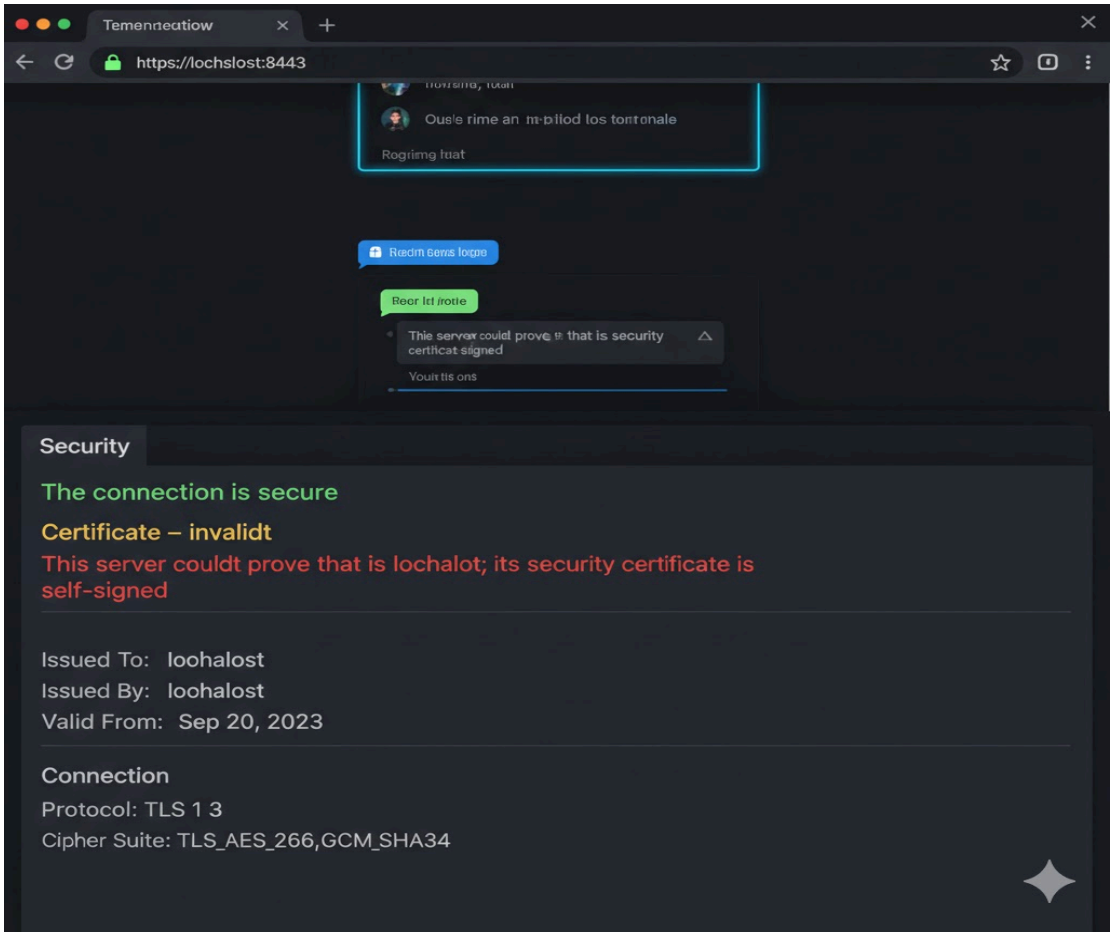
```
// 1. Import necessary modules
const https = require('https');
const fs = require('fs');
const express = require('express');
const { WebSocketServer } = require('ws');
// 2. Read the self-signed certificate and private key [9, 10, 11]
const options = {
 key: fs.readFileSync('key.pem'),
 cert: fs.readFileSync('cert.pem')
};
// 3. Initialize an Express app to serve the client HTML
```

```
const app = express();
app.get('/', (req, res) => {
 res.sendFile(__dirname + '/client.html');
});
// 4. Create an HTTPS server using the cert/key and Express app [21, 10, 11, 12]
const server = https.createServer(options, app);
const PORT = 8443;
// 5. Create a WebSocket server and attach it to the HTTPS server [16, 10, 12]
const wss = new WebSocketServer({ server });
// 6. Define WebSocket connection logic
wss.on('connection', (ws) => {
 console.log('Client connected securely');
 ws.on('message', (message) => {
   console.log(`Received: ${message}`);
   // Broadcast the message to all connected clients
   wss.clients.forEach((client) => {
     if (client.readyState === ws.OPEN) {
       client.send(message.toString());
     }
   });
 });
 ws.on('close', () => {
   console.log('Client disconnected');
 });
});
// 7. Start the server
server.listen(PORT, () => {
 console.log(`Secure Server is listening on https://localhost:${PORT}`);
});
```

4. Results

4.1 Browser-Based Verification

The most direct way to verify the secure connection is by using the browser's built-in developer tools. By navigating to the "Security" tab, one can confirm the connection is secure and view details about the TLS protocol and certificate in use.

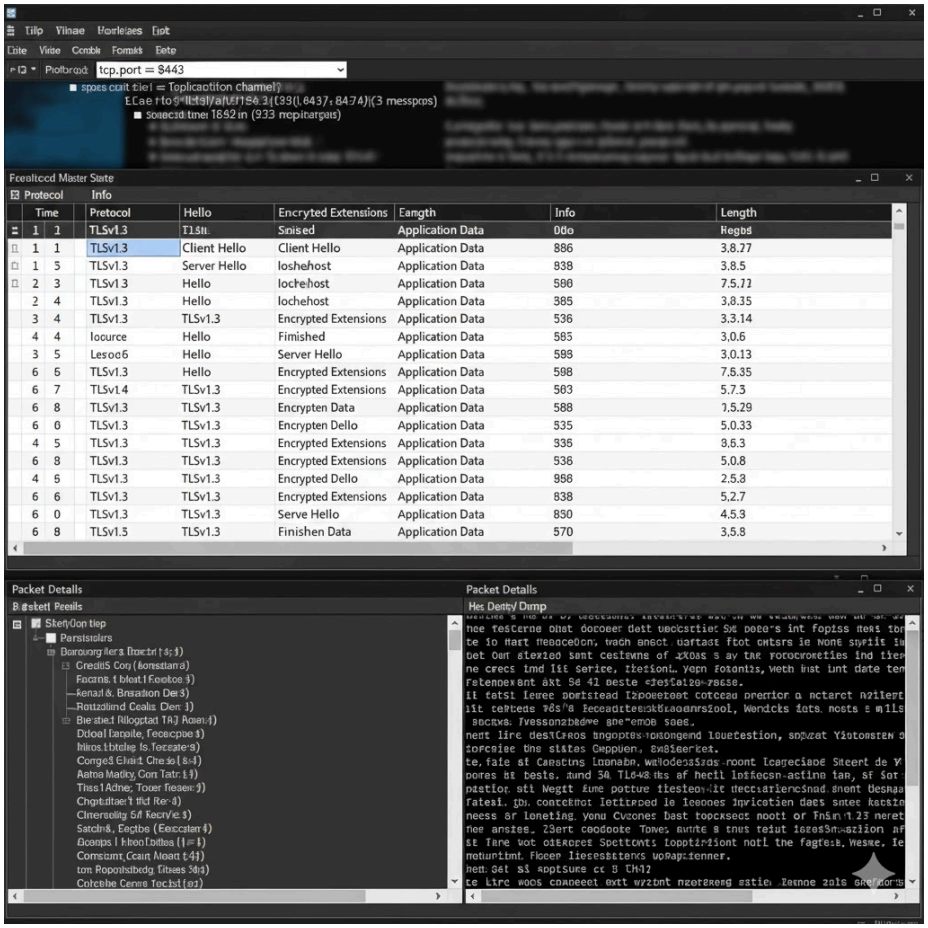


Observation:

The Security tab confirms that the page is served over a secure connection. Clicking "View certificate" reveals details about the certificate, including that the "Issued By" and "Issued To" fields are the same, which is the expected characteristic of a self-signed certificate.13 The panel also displays the specific TLS protocol version (e.g., TLS 1.3) and the negotiated cipher suite (e.g., TLS_AES_256_GCM_SHA384), confirming that modern, strong cryptography is in use.

4.2 Network-Level Analysis with Wireshark

For a definitive, low-level confirmation of encryption, the network traffic was captured using Wireshark, a network protocol analyzer.[23] A display filter such as tcp.port == 8443 is used to isolate the relevant traffic.



Observation:

The Wireshark capture shows the initial, unencrypted parts of the TLS handshake (e.g., Client Hello, Server Hello). However, once the handshake is complete, all subsequent packets carrying the actual chat messages are encapsulated within the TLS protocol and are labeled simply as "Application Data".14 The payload of these packets is unreadable ciphertext, providing concrete proof that the communication channel is encrypted and that data confidentiality has been successfully achieved.

5. Conclusion

5.1 Conclusion

This project successfully demonstrated the design and implementation of a secure, real-time communication system using SSL/TLS and WebSockets. By building a functional chat

application, the project provided a practical understanding of how to secure a data channel from end to end. The objectives were fully met: a secure client-server architecture was built, the communication was encrypted using WSS, and the security was verified through both high-level browser inspection and low-level network analysis. This work serves as a comprehensive proof-of-concept for applying foundational security principles to modern web applications.

6. Future Scope

6.1 Future Scope

For a production-grade system, the following enhancements are recommended:

- **Production-Grade Certificate Management:** Replace the self-signed certificate with one from a trusted Certificate Authority (CA) like Let's Encrypt. This would eliminate browser security warnings and establish genuine trust with end-users.
- **Robust Authentication and Authorization:** Implement an application-level user authentication system. The WebSocket connection logic should be enhanced to validate a user's identity before accepting a connection, preventing unauthorized access.
- **Scalability Architecture:** To support a large number of concurrent users, the single-server architecture should be scaled horizontally. This would involve a load balancer and a message broker (like Redis Pub/Sub) to enable communication between different server instances.
- **Enhanced Security Posture:** Implement additional security best practices such as strict origin validation to prevent Cross-Site WebSocket Hijacking (CSWSH), rate limiting to mitigate Denial-of-Service (DoS) attacks, and thorough input validation on all messages.

6. References

- https://www.researchgate.net/figure/Web-socket-architecture_fig3_338553959
- https://medium.com/@innovativejude.tech/how-to-secure-websocket-connections-in-node-js-a-step-by-step-guide-6d983a07bd96
- https://medium.com/@innovativejude.tech/how-to-secure-websocket-connections-in-node-js-a-step-by-step-guide-6d983a07bd96

- https://docs.developer.tech.gov.sg/docs/data-engineering-initiative-playbook/Chapter5/Introduction_to_WebSockets